Politecnico di Milano

School of Civil, Environmental and Land Management Engineering
MSc in Geoinformatics Engineering

Layer Alterator:

A Raster Processing Tool for Urban Change Simulation

Author: Amirhossein Donyadidegan

Supervisor: Dr. Daniele Oxoli

GeoInformatics Project (GIP) – Final Report

Academic Year 2024–2025

July 2025

Contents

1 Introduction 3

2 Background 3

3 User Manual 3
- 3.1 Overview of the Architecture . . . 3
- 3.2 Input Data Handling . . . 4
  - 3.2.1 Vector Mask . . . 4
  - 3.2.2 Raster Layers . . . 5
  - 3.2.3 Operation Rule . . . 7
  - 3.2.4 Output Folder . . . 7
- 3.3 Consistency Checks and Validation . . . 7
  - 3.3.1 CRS Alignment . . . 8
  - 3.3.2 Attribute Completeness and Validity . . . 8
  - 3.3.3 Normalization and NoData Preservation . . . 9
- 3.4 Rule Logic Overview . . . 9
  - 3.4.1 Parsing the Rules . . . 9
  - 3.4.2 Applying None Rules (C0) . . . 10
  - 3.4.3 Applying Mask Rules (C1) . . . 10
  - 3.4.4 Applying Percentage Rules (C2/C3) . . . 11
- 3.5 Output Generation . . . 13
  - 3.5.1 Naming Convention . . . 13
  - 3.5.2 Output Directory . . . 13
  - 3.5.3 Metadata and Fidelity . . . 14
  - 3.5.4 Example . . . 14
- 3.6 Rule Logic and Execution Flow . . . 14
- 3.7 Limitations and Assumptions . . . 16
  - 3.7.1 No Raster Resampling . . . 16
  - 3.7.2 Memory Usage on Large Datasets . . . 16
  - 3.7.3 No Batch Rule File Support . . . 16
  - 3.7.4 No GUI or API Interface . . . 16

4 Innovation 16

5 Conclusions 17

6 References 17

A Technical Tools and Architecture 18
- A.1 Software Tools . . . 18
- A.2 Functional Architecture . . . 18

B Testing and Debugging 19
B.1 Jupyter-Based Test Suite 19
B.2 Manual Inspection and Visual Checks 20
B.3 Debugging Strategy 20

C Demonstration: Simulating a New Highway in Milan 20
C.1 Scenario Setup 21
C.2 Vector and Rule Con guration 21
C.3 Execution and Output 22
C.4 Results 22

D Repository and Access 23

1 Introduction

Urban planners and researchers increasingly rely on digital tools to test climate adaptation strategies such as reforestation or densi cation. The Layer Alterator is a lightweight Python tool developed within the GeoInformatics Project (GIP) at Politecnico di Milano, under the supervision of Dr. Daniele Oxoli. It allows users to simulate localized changes in raster datasets based on vector-de ned rules, enabling rapid prototyping of urban interventions.

Designed for integration into urban Digital Twin work ows, the tool supports both absolute and percentage-based updates to raster layers, such as Urban Canopy Parameters (UCPs) and land cover fractions. It includes built-in validation, modular functions, and compatibility with open-source Python libraries likerasterio , geopandas, and numpy The interface is optimized for use in Jupyter Notebooks, ensuring transparency and reproducibility in simulation work ows.

2 Background

Urban simulation tools like the Local Climate Zone (LCZ) framework and high-resolution climate models have advanced our understanding of how land cover a ects climate. However, these tools often require complex data and computing resources, and are typically focused on classi cation or full physical modeling.

The Layer Alterator was developed to address a more practical need: allowing users to easily test and apply spatial changes to raster datasets using rule-based vector inputs. It lls a gap between heavy simulation platforms and static data preprocessing tools by providing a fast, modular way to create alternative raster layers for use in scenario analysis.

3 User Manual

3.1 Overview of the Architecture

The Layer Alterator system is designed with a modular architecture that follows a precise and reproducible data processing pipeline. The goal of this architecture is to ensure the separation of concerns across components, making the codebase easier to maintain, extend, and test.

The work ow is composed of four primary stages:

1. Input Data Loading : Vector masks, raster layers (UCPs and land cover fractions), and a JSON rule le are loaded using GeoPandas, Rasterio, and Python's standard libraries.

2. Rule Parsing and Classi cation : The JSON rule le is interpreted to assign operations (e.g., masking, percentage changes, or ignoring) to each raster layer. The polygon-level attributes are validated, and a rule category (C0{C5) is assigned to guide the processing logic.

3. Raster Processing : Depending on the rule classification, either masking or percentage-based updates are applied. The core processing is done using NumPy and Rasterio.

4. Output Generation : Modified raster layers are written to disk with clear naming conventions, and validation checks are enforced to ensure consistency (e.g., normalized fractions).

This structure enables the user to simulate spatially localized changes by simply updating a vector file and its associated attributes. The tool automatically adapts its logic based on the declared rule set, removing the need for manual reprogramming.

Each major block is encapsulated into reusable functions, which are documented and testable. Error handling and validation are embedded at each phase to ensure robustness and clarity during runtime.

A central control function named `layer_alterator()` coordinates the end-to-end execution. It accepts the paths to the input files and directories, loads and parses the necessary elements, determines the rule category, and triggers the appropriate set of update functions.

The resulting system strikes a balance between simplicity and extensibility, being both lightweight enough for use in exploratory simulations and modular enough to be integrated into broader modeling workflows, such as Digital Twin platforms.

3.2 Input Data Handling

The system requires three main inputs: a vector mask file, raster layers categorized into Urban Canopy Parameters (UCPs) and land cover fractions, and a JSON rule file. These inputs are used to define the spatial regions of interest and the transformations to be applied to them.

3.2.1 Vector Mask

The vector file defines the zones (typically polygons) over which operations will be applied. Supported formats include:

- `.gpkg` (GeoPackage)
- `.geojson` or `.json` (GeoJSON)
- `.shp` (Shapefile)

GeoPandas is used to read and parse the vector file into a `GeoDataFrame` (`named gdf_mask`). The system auto-detects the format based on the file extension.

Each polygon must contain numeric attributes such as `HMD`, `BSF`, `F_AC`, or `F_UF`, which represent specific values or percentage changes depending on the rule type.

Figure 1: Input vector mask example over the Milan metropolitan area

3.2.2 Raster Layers

Raster inputs are divided into two subfolders, corresponding to the types of spatial covariates used in urban climate modeling.All raster layers are normalized to the range [0, 1] , either as individual continuous variables (in the case of UCPs) or as fractional components that sum to one (in the case of land cover fractions).

- ./ucps/ { Urban Canopy Parameters (UCPs) such as:
  - IMD.tif : Impervious Surface Density { proportion of sealed surfaces
  - BSF.tif : Building Surface Fraction { fraction of ground covered by buildings
  - BH.tif : Building Height { normalized average height of buildings
  - SVF.tif : Sky View Factor { the proportion of sky visible from ground level
  - TCH.tif : Tree Canopy Height { average height of vegetation cover
- ./lc_fractions/ { Fractional land cover layers derived via spectral unmixing:
  - F_AC.tif : Asphalt/Concrete surfaces
  - F_S.tif : Shingle roofs or gravel
  - F_M.tif : Metal surfaces (e.g., rooftops)
  - F_BS.tif : Bare Soil/Sand
  - F_G.tif : Grass and low vegetation
  - F_TV.tif : Tall Vegetation (e.g., trees, forest patches)
  - F_W.tif : Water bodies

The naming convention enables the system to automatically identify and categorize raster layers. Files pre xed with F_ are interpreted as land cover fractions, while the others are treated as UCP layers.

Each raster layer is loaded using Rasterio and is expected to conform to the following criteria:

- Must match the Coordinate Reference System (CRS) of the vector input.
- Should have consistent resolution and extent across all layers (spatial alignment is assumed).
- Must include properly de ned NoData values to avoid corruption of unde ned or masked regions.

Land cover fraction layers are obtained using regression-based spectral unmixing techniques (Okujeni et al., 2016), ensuring non-negativity and pixel-wise normalization (fractions sum to 1). This makes them suitable for simulating urban interventions by adjusting proportions.

F_AC.tif { Asphalt/Concrete fraction derived from spectral unmixing over Milan

BH.tif { Normalized Building Height layer over the Milan metropolitan area

Figure 2: Raster Layer Examples

Urban Canopy Parameters are derived from datasets like high-resolution building footprints and digital surface models. These layers capture vital characteristics of the urban landscape that in uence surface urban heat islands (SUHIs), such as verticality, density, and vegetation structure. All UCP layers are normalized to a [0{1] range for fair weighting in machine learning and raster operations. These raster inputs act as covariates in urban climate models, enabling simulations of land surface temperature (LST) responses to urban transformations.

3.2.3 Operation Rule

A JSON le de nes how each raster layer should be processed. The supported rule types are:

- "mask" { Apply an absolute value per polygon
- "pct" { Apply a percentage change relative to existing pixel values
- "none" { Skip processing of this layer

Listing 1: Rule File Example

```
{
    "F_AC.tif": "mask",
    "F_UF.tif": "mask",
    "IMD.tif": "pct",
    "BSF.tif": "pct"
}
```

This le is parsed using Python's built-in json module and loaded into a dictionary called `rules`.

3.2.4 Output Folder

A designated directory (default: ./output/) is used to store the processed raster outputs. The system appends su xes, such as_mask.tif or _pct.tif , to indicate the transformation that has been applied. If the folder does not exist, it is created during runtime.

This structured input system makes the tool easy to con gure for new scenarios by simply updating the polygon attributes or changing the JSON rule le.

Figure 3: Recommended folder and le structure for running the Layer Alterator tool

3.3 Consistency Checks and Validation

Before any raster processing is performed, the system conducts a series of consistency checks to ensure data integrity and to prevent the propagation of errors throughout the pipeline. These validation steps are critical for both correctness and reproducibility.

3.3.1 CRS Alignment

The function check_crs_match() ensures that all raster layers (UCPs and fractions) share the exact Coordinate Reference System (CRS) as the vector mask. Each raster is opened individually, and its CRS is compared with the CRS of the input GeoDataFrame. Any mismatch triggers an error, including the file path of the offending raster.

Listing 2: Test log for check_crs_match()

```
--- Test 1: All CRS Match ---
All raster layers have the same CRS as the vector mask.

--- Test 2: All Rasters Match CRS but Different from Vector ---
Warning: CRS mismatch detected in the following layers:
 - BH.tif: CRS = EPSG:32632, differs from vector CRS = EPSG:7415
 - F_AC.tif: CRS = EPSG:32632, differs from vector CRS = EPSG:7415

--- Test 3: CRS Mismatch ---
Warning: Could not open raster for layer IMD_differentCRS.tif: test_data/fun_2/
    IMD_differentCRS.tif: No such file or directory
All raster layers have the same CRS as the vector mask.

--- Test 4: Missing Raster File ---
Warning: Could not open raster for layer nonexistent.tif: test_data/fun_2/nonexistent.tif:
    No such file or directory
All raster layers have the same CRS as the vector mask.
```

3.3.2 Attribute Completeness and Validity

For mask and pct operations, the system verifies that all attributes required by the rules are present in every polygon of the vector mask. If any raster is to be updated using a missing attribute, execution stops and the user is notified with an error message.

In C1 mode, the following logical checks are also enforced:

- IMD must be greater than or equal to BSF, i.e., IMD $\geq$ BSF.
- The sum of all land cover fractions with prefix F_ must equal 1.0 for each polygon.

Figure 4: BH.tif – Normalized Building Height layer over the Milan metropolitan area

Listing 3: Test log for check_imd_bsf_consistency()

```
Warning: 259257 pixels (2.94%) have IMD < BSF.
Consider adjusting the 'pct' attributes in the vector mask.
```

In C2 and C3 modes, a similar check is applied on a per-pixel basis after the percentage changes are applied. The system ensures that fraction rasters still sum to 1.0 at each location and that no negative values occur. If normalization is not mathematically feasible, the tool raises a warning and advises the user to revise the input values.

3.3.3 Normalization and NoData Preservation

During raster updates, the tool carefully avoids modifying pixels that have been flagged as NoData in the input rasters. All update logic (whether masking or percentage change) uses logical masks to isolate only valid, non-null pixels. This ensures that undefined areas are preserved in the output.

For fractional layers, normalization is applied only to pixels fully covered by at least one polygon. All changes are aggregated across polygons before normalization to ensure correct behavior in overlapping zones.

These consistency checks and validation steps are crucial for producing reliable and interpretable results, particularly in urban modeling workflows where even minor errors can propagate into significant inaccuracies in downstream simulations.

Listing 4: Test log for parserules_from_mask()

```
--- Test 4: Mix of pct and none ---
Rule C3: Layers set to PCT and NONE. Proceeding with PCT logic and treating NONE as 0.

--- Test 5: Mix of mask and none  ---
Rule C4 violation: MASKING + NONE is not allowed.

--- Test 6: Mix of mask and pct ---
Rule C5 violation: MASKING + PCT is not allowed.
```

3.4 Rule Logic Overview

The Layer Alterator modifies raster layers based on vector-defined rules provided through a JSON configuration file. Each raster can be assigned a different mode of operation (`mask`, `pct`, or `none`) allowing the simulation of fixed or proportional changes across spatial units.

The logic is organized into three main stages: **Parsing Rules**, **Applying Mask Rules**, and **Applying Percentage Rules**.

3.4.1 Parsing the Rules

Each raster listed in the JSON rule file is associated with one of the following operations:

- `mask`: assign a fixed value from the polygon attribute directly to the raster pixels.
- `pct`: apply a percentage-based increase or decrease to the existing raster values.
- `none`: exclude the layer from any updates.

The system validates these assignments and categorizes the rule configuration into one of six classes:

- C0: All layers set to none (valid but no changes applied).
- C1: All layers set to mask (valid).
- C2: All layers set to pct (valid).
- C3: Mix of pct and none (valid).
- C4: Mix of mask and none (invalid).
- C5: Mix of mask and pct (invalid).

C4 and C5 raise errors and halt processing. The rule classification is essential for selecting the proper transformation logic.

3.4.2 Applying None Rules (C0)

This configuration is allowed and confirms that the rule file is syntactically valid, but it results in no raster updates. It can be used for testing the pipeline setup or deliberately skipping modification in a simulation run.

3.4.3 Applying Mask Rules (C1)

When all layers are marked as mask, the simulator performs a deterministic update. For each polygon in the vector mask, the raster values are overwritten by the corresponding attribute values.

This mode is suitable for representing complete redevelopment or fixed land cover targets.

Validation Requirements:

Each polygon must include all required attributes (e.g., IMD, BSF, and fraction components like F_AC, F_G, etc.).

Per-Polygon Constraints:

For class C1, the following conditions must hold for each feature in the vector layer:

- C1.1: Attribute Presence | All necessary attributes defined in the rule file must be present in every polygon. These include both absolute layers (e.g., IMD, BSF) and fractional layers (e.g., F_AC, F_G, etc.).
- C1.2: Built Space Constraint | The intensity of mixed development (IMD) must be greater than or equal to the built surface fraction (BSF):

$$IMD \quad BSF$$

- C1.3: Fractional Sum Constraint | The sum of all land cover fractions must equal exactly 1.0 (within a defined numerical tolerance), ensuring a consistent and closed representation of land use:

$$\sum_{F_x} F_x = 1.0$$

Listing 5: Test log for C1

```
--- Test 2: All mask ---
Rule C1: All layers set to MASKING. Checking per-polygon constraints...

Checking Feature ID 0
IMD: 0.9, BSF: 0.8
Fraction values and sum:
 Feature ID Fraction Key  Value
          0         F_AC    0.7
          0          F_S    0.0
          0          F_M    0.2
          0         F_BS    0.0
          0          F_G    0.1
          0         F_TV    0.0
          0          F_W    0.0
          0          SUM    1.0
...
All per-polygon constraints (C1.1, C1.2, C1.3) are satisfied.
```

Output: Masked rasters are saved using the suffix _mask.tif, allowing clear identification of deterministic changes.

Figure 5: Visualization of the masking operation on F_AC.tif.

3.4.4 Applying Percentage Rules (C2/C3)

If the rule category is C2 or C3, the system performs percentage-based updates to raster values using polygon-level attributes. These updates apply to either UCP (Urban Change Parameters) or land cover fraction layers, depending on the rule file.

UCP Layers:

- Each pixel within a polygon is multiplied by a scaling factor derived from the attribute: $\text{factor} = 1 + \frac{\text{pct_value}}{100}$.
- Pixels with a value of 0.0 are, by default, replaced with a small positive value (e.g., 0.01) before scaling to avoid null propagation. This behavior is controlled by the zero_handling parameter.
- If the polygon attribute is null or missing, a value of 0.0 (i.e., no change) is used.
- NoData pixels in the raster are detected and preserved throughout the process.

- After all modifications, the output is checked for values outside the valid range [0, 1]:
  - If clip mode is enabled, values above 1.0 are truncated to 1.0, and below 0.0 to 0.0.
  - If normalize is enabled, values are rescaled linearly across $[0, 1]$.
  - If ignore is set, values are left unchanged.

Listing 6: Test log for C1

```
--- Test 3: All pct ---
Rule C2: All layers set to PCT. Proceeding with PCT logic.

--- Test 4: Mix of pct and none ---
Rule C3: Layers set to PCT and NONE. Proceeding with PCT logic and
    treating NONE as 0.
```

Figure 6: Visualization of the percentage operation on MD.tif .

Fraction Layers:

- Each raster representing a fraction (e.g. F_G, F_AG) is scaled by its corresponding percentage change from the polygon attribute.
- After scaling, all fractions at each pixel are re-normalized to ensure their sum equals 1.0.
- If normalization fails (e.g., due to all-zero or negative values), the system resets the fractions to default (e.g., uniform split) and logs a warning for review.
- Null or missing attributes are treated as 0 (no change).

Output: All percentage-modified rasters are saved with the suffix_pct.tif, enabling traceability across simulations.

Together, these rule modes support absolute and incremental urban change modeling with flexible control over numerical safety and error handling.

Figure 7: Visualization of the percentage operation on F_AC.tif.

3.5 Output Generation

The final phase of the processing workflow involves exporting the updated raster layers to disk. This step ensures that each transformation applied during masking or percentage operations is traceable and organized.

3.5.1 Naming Convention

The system uses suffixes in the output filenames to indicate the type of transformation applied:

- Rasters updated withmaskrules receive a_mask.tif suffix.
- Rasters updated withpct rules receive a_pct.tif suffix.

This makes it easy for users to identify the nature of the changes at a glance, mainly when multiple simulation scenarios are run in sequence.

3.5.2 Output Directory

All processed rasters are written to a user-defined output folder (default:/output/). If the folder does not exist, it is created automatically. The code preserves the original filenames, adding the relevant suffix so that users can trace each output back to its source raster.

3.5.3 Metadata and Fidelity

Each output raster retains the spatial resolution, coordinate system (CRS), and NoData de nition of its original input. Only the pixel values are modi ed according to the rules and attributes. This ensures seamless integration into subsequent geospatial work ows (e.g., visualization, simulation, or model training).

3.5.4 Example

Given an input raster F_AC.tif with a pct rule applied, the output will be written as:

./output/F _ACpct.tif

If the same le were processed with a mask rule instead, the result would be:

./output/F _ACmask.tif

This explicit output structure enhances traceability, making it easier to evaluate and compare di erent simulation outputs.

3.6 Rule Logic and Execution Flow

The internal rule engine of the Layer Alterator tool processes raster layers based on rule types assigned in a user-provided JSON le. Each layer is classi ed as either "mask", "pct" , or "none" type, and undergoes processing accordingly.

Figure 8 illustrates the complete decision logic pipeline, from parsing the rule le to applying the corresponding operation on each raster layer.

Figure 8: Flowchart summarizing rule classi cation and execution logic in the Layer Alterator tool.

3.7 Limitations and Assumptions

While the Layer Alterator provides a flexible and robust framework for modifying raster data using vector-defined rules, several limitations and assumptions currently apply:

3.7.1 No Raster Resampling

Rasters must have the exact spatial resolution and grid alignment. The system does not resample rasters or align grids; it assumes pre-aligned inputs. Inconsistent raster dimensions may result in runtime errors or incorrect masking behavior.

3.7.2 Memory Usage on Large Datasets

The current implementation loads all fraction layers into memory during the normalization process. While this improves performance for moderate-sized rasters, it can result in memory limitations for vast study areas. A tiled or streaming implementation is suggested for future scalability.

3.7.3 No Batch Rule File Support

Only one JSON file is accepted per run. Batch simulations with different rule sets require multiple separate executions. Incorporating a batch-processing feature could enable automated testing across many scenarios.

3.7.4 No GUI or API Interface

The tool is designed to be run as a script or within a Jupyter notebook. There is no graphical interface or web API available at present; however, its modularity allows for integration into external applications or Digital Twin interfaces.

4 Innovation

The **Layer Alterator** offers a lightweight and flexible solution for simulating spatial changes in urban environments. Unlike traditional workflows that rely on heavy classification or modeling systems, this tool allows users to apply custom rules directly to raster datasets using a simple vector file with polygon-level attributes.

Its main innovation is the ability to handle multiple raster layers in a single run, combining both absolute updates (masking) and relative changes (percentages). This makes it easy to simulate complex, localized interventions such as increasing vegetation or adjusting built-up density—without modifying the code.

Additional built-in features, such as automated CRS validation, rule consistency checks, and fractional normalization, ensure that results are reliable and ready for use in further simulations or visualizations. The modular Python implementation also supports integration with Digital Twin platforms or other urban planning tools, making it suitable for both research and policy applicat

5 Conclusions

The Layer Alterator project successfully delivers a prototype application capable of simulating spatially-explicit changes in raster datasets using polygon-based vector rules. It achieves its goal of providing a lightweight, transparent, and adaptable tool for urban simulation scenarios, with particular relevance to climate-sensitive domains such as Urban Heat Island (UHI) modeling.

The modular architecture, use of open-source Python libraries, and Jupyter-integrated testing environment make the tool highly extensible. Its ability to validate, classify, and apply transformations based on rule-driven logic demonstrates strong potential for integration into Digital Twin platforms.

Through extensive testing and validation, the tool proved reliable in both deterministic (`mask`) and proportional (`pct`) update scenarios. Output rasters consistently preserved CRS, NoData, and normalization requirements, ensuring compatibility with downstream analytical workflows.

Future development could extend functionality through batch scenario support, GUI/web integration, and memory optimization for large-scale raster processing. Overall, the Layer Alterator serves as a valuable building block for interactive geospatial modeling, empowering stakeholders to test "what-if" urban interventions with minimal technical overhead.

6 References

1. Stewart, I. D., & Oke, T. R. (2012). Local Climate Zones for Urban Temperature Studies. *Bulletin of the American Meteorological Society*, 93(12), 1879–1900. https://doi.org/10.1175/BAMS-D-11-00019.1

2. Okujeni, A., van der Linden, S., Sitnikova, D., & Hostert, P. (2016). Mapping Urban Land Cover Based on Fractional Abundances from Impervious Surface Time Series. *Remote Sensing of Environment*, 187, 1–15. https://doi.org/10.1016/j.rse.2016.10.002

3. Vavassori, A., Oxoli, D., Venuti, G., Brovelli, M. A., Siciliani de Cumis, M., Sacco, P., & Tapete, D. (2024). A Combined Remote Sensing and GIS-Based Method for Local Climate Zone Mapping Using PRISMA and Sentinel-2 Imagery. *International Journal of Applied Earth Observation and Geoinformation*, 131, 103944. https://doi.org/10.1016/j.jag.2024.103944

A Technical Tools and Architecture

The development of the **Layer Alterator** tool relies entirely on open-source Python libraries widely used in geospatial data processing and numerical computation. The following tools form the technical backbone of the system:

A.1 Software Tools

- **Python 3.11**: Core programming language used for all functions.
- **Jupyter Notebook**: Used for interactive execution, testing, and demonstration.
- **GeoPandas**: For loading and manipulating vector data (e.g., masks).
- **Rasterio**: For raster I/O operations, masking, and value access.
- **NumPy**: For fast raster value manipulation, logical masking, and normalization.
- **Pandas**: For attribute table parsing and validation.
- **Shapely**: For geometric operations and polygon rasterization.

All dependencies are listed in a `requirements.txt` file and managed via a dedicated virtual environment.

A.2 Functional Architecture

The system is organized into modular Python functions, each handling a specific processing task. Core components include:

- `load_vector_mask()`: Loads polygon layers from GPKG, GeoJSON, or Shapefiles.
- `check_crs_match()`: Verifies that all input rasters match the vector CRS.
- `parse_rules_from_mask()`: Loads and classifies the rule JSON into C0–C5 categories.
- `apply_masking()`: Applies fixed values from vector attributes to raster pixels.
- `apply_mask_rule_all()`: Executes masking across all layers marked as `mask`.
- `is_fraction_layer()`: Identifies whether a layer is a fractional land cover raster.
- `apply_pct_ucp()`: Applies percentage changes to UCP layers.
- `apply_pct_all_fractions()`: Scales and normalizes land cover fraction layers.
- `apply_pct_all()`: Orchestrates updates across all layers based on `pct` rules.
- `check_imd_bsf_consistency()`: Ensures `IMD` `BSF` in updated rasters.

- `layer_alterator()`: Main controller function for rule validation and execution.

All functions are modular, documented, and tested within a Jupyter Notebook environment.

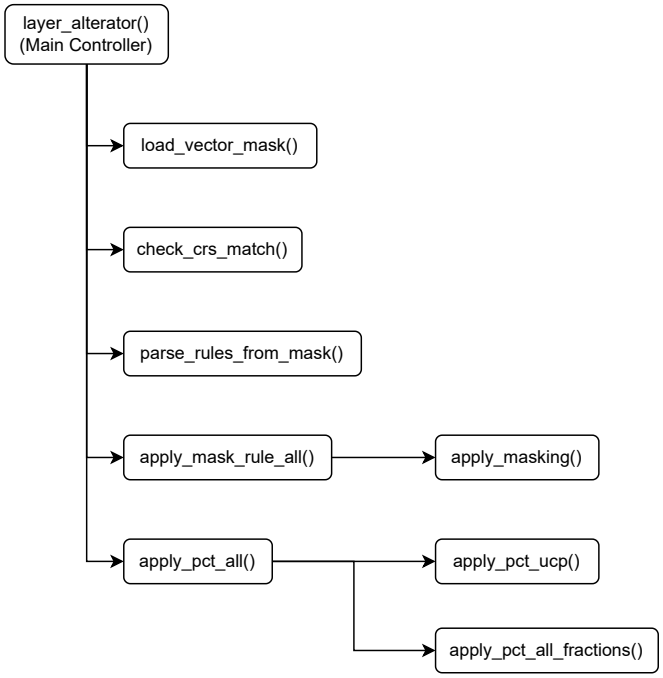

Figure 9: Modular architecture of the Layer Alterator tool showing the relationship between core functions.

B Testing and Debugging

Extensive testing was carried out to validate the correctness, reliability, and robustness of the **Layer Alterator** tool. Testing activities focused on verifying individual functions, integrating across modules, and validating results with both synthetic and real geospatial data.

B.1 Jupyter-Based Test Suite

A dedicated Jupyter notebook (`test_functions.ipynb`) was developed to test each processing function in isolation. These include:

- CRS validation with intentionally mismatched inputs
- Masking behavior with varying polygon geometries and attribute values
- Percentage update logic and normalization for multiple fraction rasters

- Error handling for edge cases (e.g., missing attributes, empty geometries, invalid rule types)

Each test case was designed to produce a verifiable result, either by checking raster outputs directly or through controlled assertions embedded in the notebook.

B.2 Manual Inspection and Visual Checks

Raster outputs were also verified visually using QGIS. Particular attention was paid to:

- Alignment between polygon masks and updated raster regions
- Correct propagation of changes within and across overlapping polygons
- Integrity of NoData pixels post-processing
- Visual sum-to-one verification of updated fraction layers

These manual inspections helped ensure that the system performed as expected in spatial and visual terms.

B.3 Debugging Strategy

Debugging was facilitated by:

- Extensive use of inline print statements and logging for intermediate values
- Isolating raster subsets (e.g., single tile) for step-by-step tracing

Common issues encountered included CRS mismatches, shape misalignments, and incorrect fraction normalization in cases involving overlapping polygons. All problems were resolved by refining the validation logic and adjusting the masking strategies.

This rigorous testing and debugging phase was essential to delivering a stable and reusable tool suitable for inclusion in analytical pipelines or Digital Twin demonstrators.

C Demonstration: Simulating a New Highway in Milan

To demonstrate the capabilities of the **Layer Alterator** tool, we designed a realistic scenario in which a new highway is introduced in the northern part of the Milan metropolitan area. This simulation showcases how the tool applies vector-defined masking rules to update raster layers representing urban surface characteristics.